

PROJECT: AWS Automated Backup and Restore System

Services: AWS Cost Explorer, AWS Budgets, Amazon SNS, AWS Lambda, Amazon SES, AWS IAM, Amazon CloudWatch.

Project Goal: The primary goal of this project is to prevent unexpected AWS billing overruns by providing continuous cost visibility, proactive budget enforcement, and automated alerting without manual intervention.

Skills: AWS Cost Management, Serverless Architecture, Event-Driven Automation, Cloud Financial Governance, IAM Security, Cost Optimization.

Definition: AWS Cost Monitoring & Budget Alerts is a serverless cloud cost governance project designed to track AWS spending in real time, evaluate actual and forecasted costs against predefined budgets, and automatically notify stakeholders when budget thresholds are exceeded using event-driven automation.

Scope of the Project: The scope of this project includes monitoring AWS service usage and costs using Cost Explorer, defining monthly budgets and threshold limits through AWS Budgets, and implementing automated alerts when spending exceeds defined limits. It covers event-driven notifications using SNS, serverless processing with Lambda, and email alerts via SES. The project also enforces secure access using IAM roles, supports cost allocation through tagging, and enables monitoring and troubleshooting using CloudWatch, ensuring scalable, secure, and efficient cloud cost management.

Methodologies:

The same AWS Automated Backup and Restore System can be implemented using the following methods:

Method 1: AWS Budgets with Direct Email Alerts

AWS Budgets can directly send email notifications when budget thresholds are exceeded.

Limitations:

- No automation or custom logic
- Limited alert customization
- Not suitable for enterprise-level workflows

Method 2: CloudWatch Billing Alarms

CloudWatch billing alarms monitor estimated charges and trigger notifications.

Limitations:

- Supports only total billing amount
- No forecasted cost analysis
- Limited cost breakdown and automation

Method 3: Third-Party Cost Management Tools

External tools like CloudHealth or CloudCheckr can be used for cost monitoring.

Limitations:

- Additional licensing cost
- External dependency
- Complex integration and configuratio

Why Our Method Is Better Compared to Others

1. Fully Automated and Event-Driven

Budget breaches automatically trigger actions without continuous monitoring, reducing cost and operational overhead.

2. High Customization

Lambda allows custom alert logic, formatted emails, and easy extension for future automation such as resource shutdowns.

3. Scalable and Serverless

The solution automatically scales with usage and requires no infrastructure management.

4. Better Cost Visibility

Supports both actual and forecasted cost tracking, unlike CloudWatch billing alarms.

5. Secure and Reliable

IAM enforces least-privilege access and AWS-native services ensure high availability and security.

6. Cost-Effective

Uses pay-as-you-go AWS services with no third-party licensing fees.

Services Used in this Project:

1. AWS Cost Explorer

Definition: AWS Cost Explorer is a cost management service that helps visualize and analyze AWS usage and spending over time.

Purpose: To provide detailed insights into AWS costs, usage patterns, and spending trends.

Role in the Project: It acts as the foundation for cost visibility by supplying historical and forecasted cost data used by AWS Budgets for monitoring and analysis.

2. AWS Budgets

Definition: AWS Budgets allows users to set custom cost and usage budgets and continuously monitor spending against defined limits.

Purpose: To enforce spending limits and detect cost overruns early.

Role in the Project: It continuously evaluates actual and forecasted AWS costs and triggers alerts when budget thresholds are exceeded.

3. Amazon Simple Notification Service (SNS)

Definition: Amazon SNS is a fully managed messaging service that enables message delivery using a publish-subscribe model.

Purpose: To reliably deliver notifications between AWS services.

Role in the Project: SNS receives budget breach notifications from AWS Budgets and forwards them to AWS Lambda for automated processing.

4. AWS Lambda

Definition: AWS Lambda is a serverless compute service that runs code in response to events without requiring server management.

Purpose: To execute automation logic only when triggered.

Role in the Project: Lambda processes budget alert messages from SNS and triggers email notifications through Amazon SES.

5. Amazon Simple Email Service (SES)

Definition: Amazon SES is a scalable email service used for sending transactional and notification emails.

Purpose: To deliver reliable and cost-effective email notifications.

Role in the Project: SES sends automated budget alert emails to administrators and stakeholders when spending exceeds limits.

6. AWS Identity and Access Management (IAM)

Definition: AWS IAM is a security service used to manage access to AWS resources through roles and policies.

Purpose: To enforce secure, least-privilege access control.

Role in the Project: IAM controls permissions for AWS Budgets, SNS, Lambda, and SES, ensuring only authorized services can access billing and notification functions.

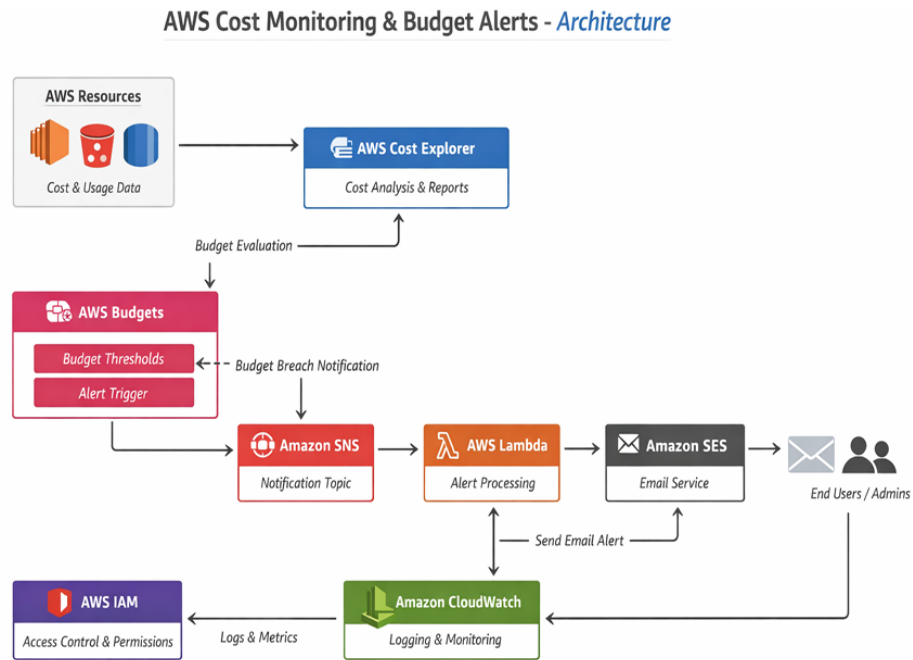
7. Amazon CloudWatch

Definition: Amazon CloudWatch is a monitoring and logging service for AWS resources and applications.

Purpose: To monitor system performance and troubleshoot issues.

Role in the Project: CloudWatch tracks Lambda execution logs, monitors errors, and helps validate the alerting workflow

Architecture Diagram



Implementation and Execution of AWS Automated Backup and Restore System .

Phase 1: Login to AWS Management Console

1. Open a web browser and navigate to the **AWS Management Console**.
2. Login using valid AWS account credentials.
3. Ensure you are logged in as a **root user**

Phase 2 : Enable AWS Cost Explorer

1. Navigate to **Billing and Cost Management**.
2. Select **Cost Explorer** from the left navigation panel.
3. Click **Enable Cost Explorer**.
4. Wait for up to **24 hours** for cost data to populate.

Purpose:

Enables visibility into AWS usage and cost data.

Phase 3: Enable Cost Allocation Tags

1. Go to **Billing → Cost Allocation Tags**.
2. Activate relevant tags (e.g., Project, Environment).
3. Save changes.

Purpose: Allows granular tracking of costs by resource or project

Cost allocation tags Info

Cost allocation tags activated: 1

[Backfill tags](#) [Download CSV](#)

How it works

Enable additional user-based cost allocation tags
Track AWS costs by user attributes from your identity provider for enhanced organizational cost analysis. Enable in Cost management preferences to use for cost allocation tags. [Cost Management Preferences](#)

Cost allocation tags (1/8) Info

1 matches

[Status = Active](#) [Clear filters](#)

☒	Tag key	Status	Type	Last updated date	Last used month
☒	Project	Active	Resource	January 31, 2026, 12:14 (UTC+05:30)	December 2025

Phase 4: Create an SNS Topic

1. Navigate to Amazon SNS.
2. Click Topics → Create topic.
3. Select Standard topic.
4. Enter topic name (e.g., aws-budget-alert-topic).
5. Click Create topic.

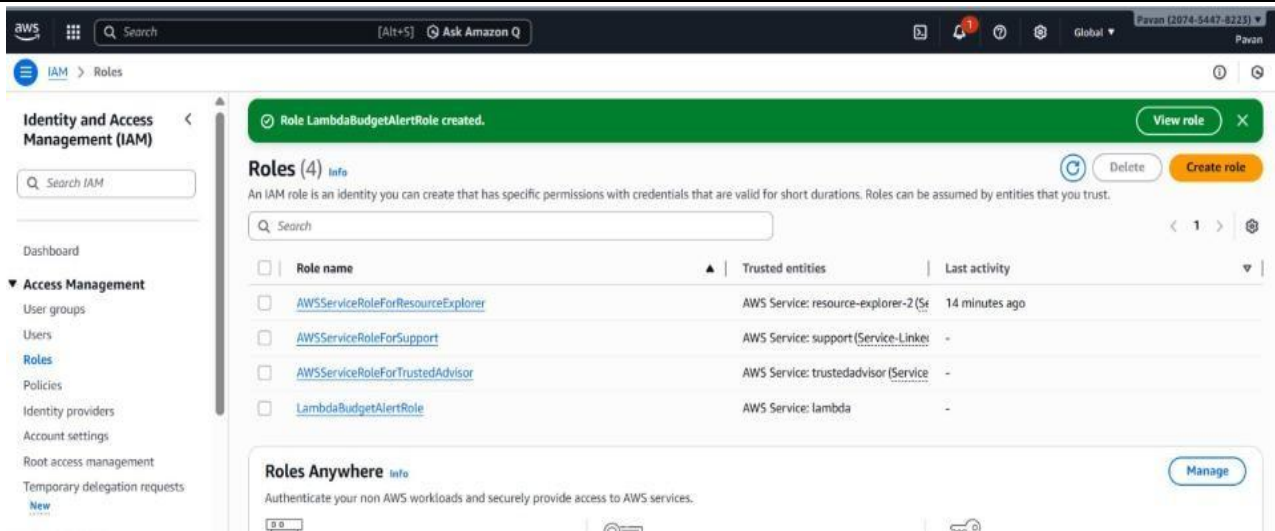
Amazon SNS **aws-budget-alert-topic** [Edit](#) [Delete](#) [Publish message](#)

Details

Name aws-budget-alert-topic	ARN arn:aws:sns:ap-south-1:207454478223:aws-budget-alert-topic	Display name -	Type Standard
Topic owner 207454478223			

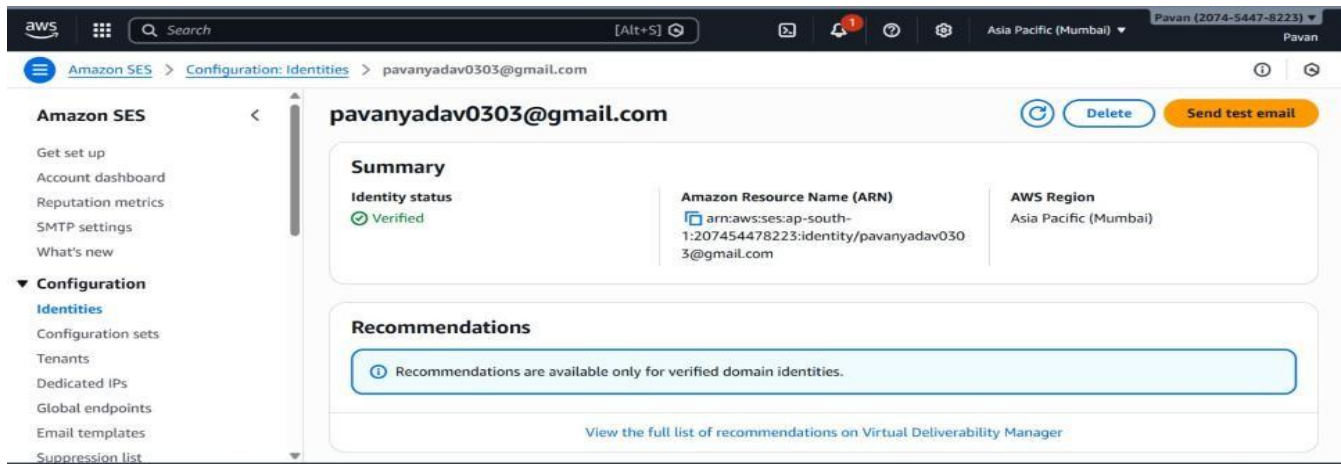
Phase 5: Create an IAM Role for Lambda

1. Navigate to IAM → Roles → Create role.
2. Select AWS service → Lambda.
3. Attach the following policies:
 - AWSLambdaBasicExecutionRole
 - AmazonSESFullAccess
 - AmazonSNSReadOnlyAccess
4. Name the role and create it.



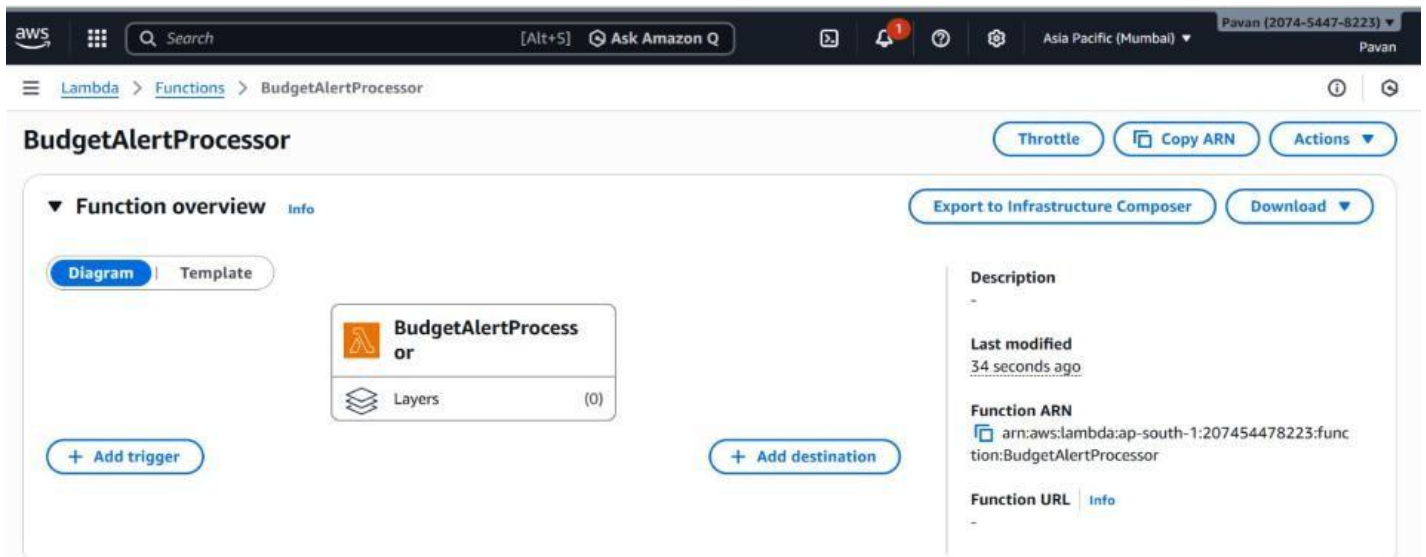
Phase 6 : Configure Amazon SES

1. Navigate to Amazon SES.
2. Click Get started.
3. Verify an email address.
4. Confirm verification through email



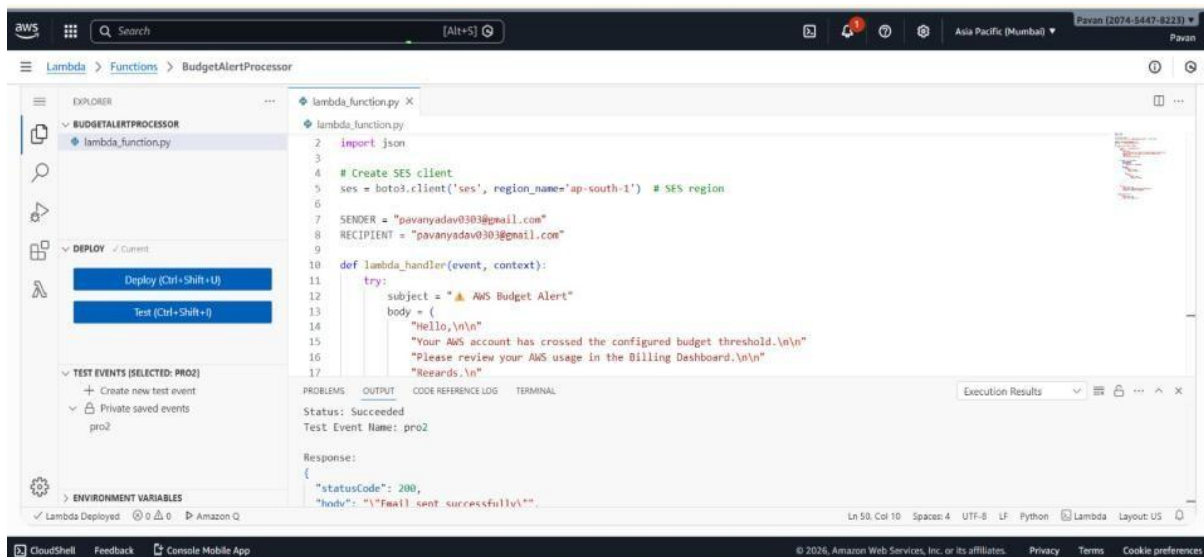
Phase 7 : Create AWS Lambda Function

1. Navigate to AWS Lambda → Create function.
2. Select Author from scratch.
3. Runtime: Python 3.x (latest available).
4. Architecture: x86_64.
5. Assign the IAM role created earlier.
6. Create the function
7. Click **Create function**



Phase 8: Write Lambda Function Code

1. Add Python code to:
 - Parse SNS message
 - Format alert content
 - Send email using SES
2. Save and deploy the function.

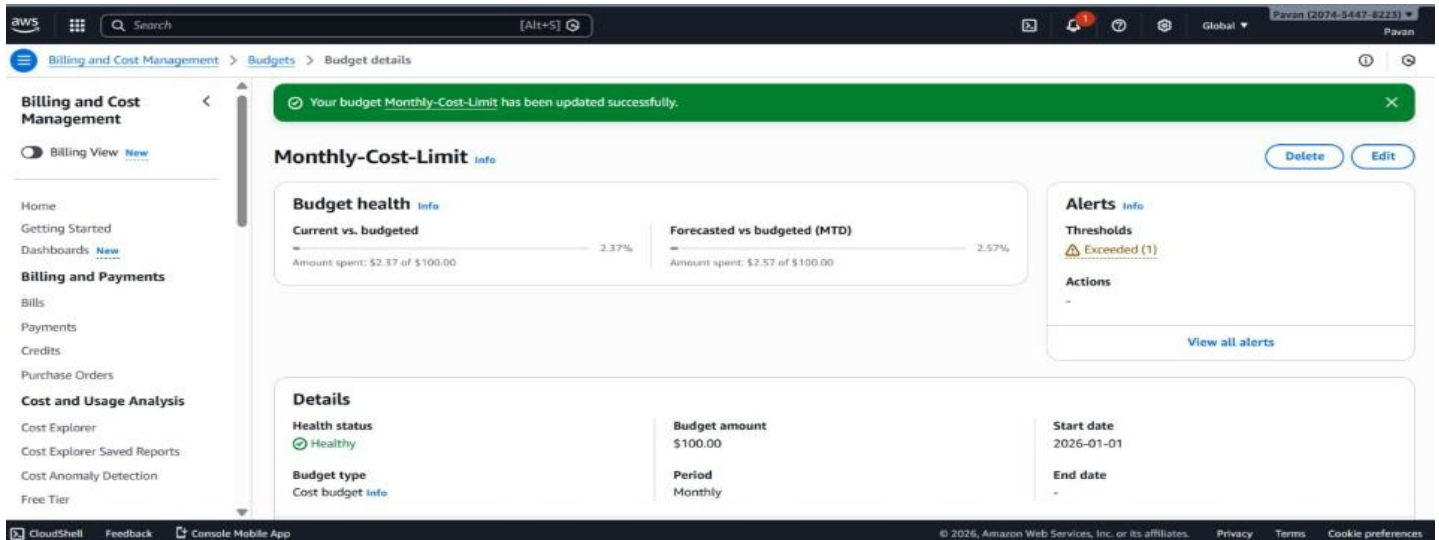


Phase 9 : Subscribe Lambda to SNS Topic

1. Open the SNS topic created earlier.
2. Click Create subscription.
3. Protocol: AWS Lambda.
4. Endpoint: Select Lambda function.
5. Confirm subscription.

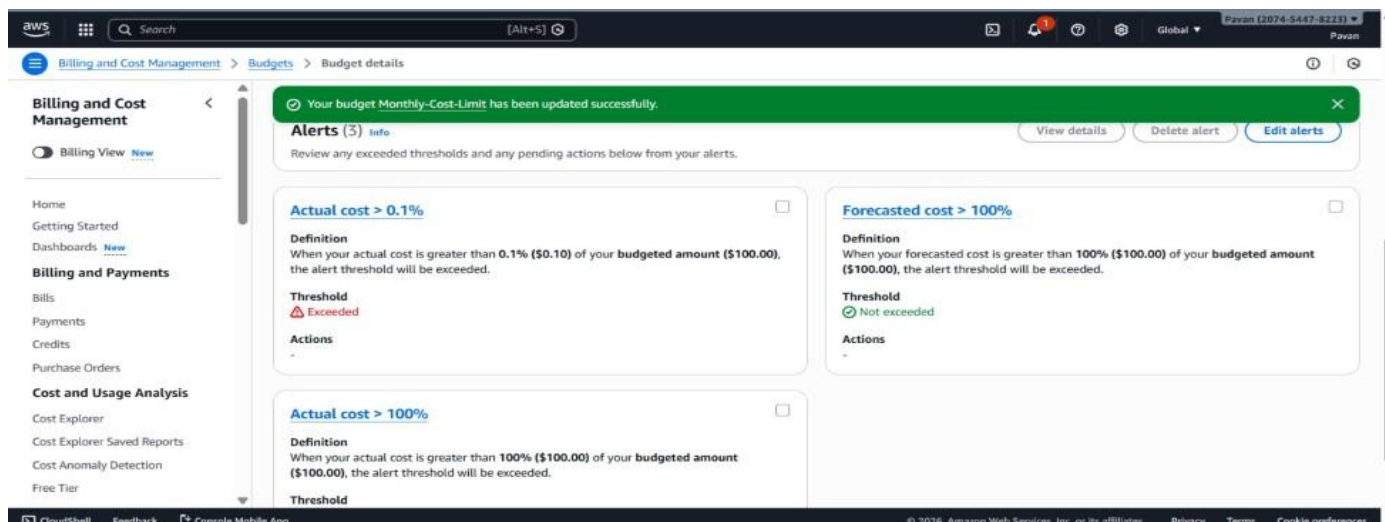
Phase 10 : Create AWS Budget

1. Navigate to Billing → AWS Budgets.
2. Click Create budget.
3. Select Cost budget.
4. Define:
 - Monthly budget amount
 - Budget period
5. Add threshold alerts (e.g., 80%, 100%).



Phase 11: Configure Budget Actions

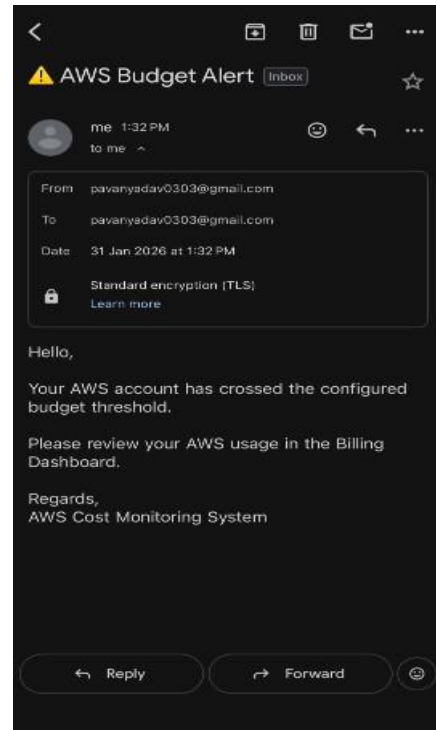
1. For each threshold:
 - Choose SNS notification.
 - Select aws-budget-alert-topic.
2. Save budget configuration.



Phase 12 : Test End-to-End Workflow

1. Launch a low-cost AWS resource (e.g., EC2).
2. Allow cost usage to accumulate.
3. Budget threshold is breached.
4. SNS triggers Lambda.
5. Lambda sends email via SES.
6. Verify email notification received.

The mail received after the threshold reach →



Troubleshooting

Issue 1: No Cost Data Visible in AWS Cost Explorer

Cause:

AWS Cost Explorer was newly enabled or there was no recent AWS resource usage.

Solution:

Waited up to 24 hours for billing data to populate and ensured at least one AWS service was actively running.

Issue 2: AWS Budgets Not Triggering Alerts

Cause:

Budget thresholds were not configured correctly or notifications were not linked to an SNS topic.

Solution:

Verified budget amount, thresholds (actual and forecasted), and correctly attached the SNS topic to each alert rule.

Issue 3: SNS Topic Created but No Notification Sent

Cause:

SNS topic had no active subscription or incorrect protocol configuration.

Solution:

Confirmed Lambda was subscribed to the SNS topic with the correct protocol and ensured the subscription status was confirmed.

Conclusion

The AWS Cost Monitoring & Budget Alerts project implements an automated and serverless approach to monitor AWS spending and control costs effectively. By using AWS Cost Explorer, AWS Budgets, Amazon SNS, AWS Lambda, and Amazon SES, the system provides real-time cost visibility and sends timely alerts when budget thresholds are crossed.

This event-driven solution reduces manual effort, improves cost awareness, and helps prevent unexpected billing overruns. It follows AWS best practices for security, scalability, and cost efficiency, making it a practical and industry-relevant solution for cloud financial governance.